VUL-FIX AGENT UPGRADE GUIDE

Step 3 · Agent Fix Intelligence — Model-Agnostic (GPT-4o / Any LLM)

Prepared from architecture discussion · June 2026

1. Purpose & Scope

This document captures every gap identified in the current vul-fix agent and the exact prompt / logic change needed to close it. All changes are confined to **Step 3 – agent fix**. The existing validate-patch, mvn verify retry loop, and push logic are untouched. The approach is model-agnostic: GPT-4o (free or paid), Claude, Gemini, or any OpenAI-compatible endpoint.

2. Identified Gaps

#	Gap	Impact
G1	Agent fixes CVEs sequentially, not holistically	Fixes conflict with each other
G2	No Spring Boot upgrade evaluation before patching	10+ CVEs that one upgrade would resolve remain
G3	Transitive deps fixed without exclusion/addition strategy	Build breaks after patch
G4	No project memory — rediscovers structure every run	Slow; repeats failed strategies
G5	Multi-module projects treated as flat single pom	Wrong pom edited; child modules missed
G6	No awareness of shared commons-jar / upstream deps	Downstream repos fail after push
G7	No Nexus availability check before downstream fix	Build fails waiting for unpublished artifact
G8	Staged rollout not enforced (canary → batch → all)	One bad fix propagates to 100+ repos

3. Project Registry (File-Based Memory)

Create **.vul-fix/project-registry.json** in each repo root. The agent reads this before acting and updates it after a successful fix. Addresses gaps G4, G5, G6.

```
{
"type": "single-pom", // or "multi-module"
"modules": ["api-core","api-web"], // multi-module only
"parentPom": "pom.xml",
"springBootVersion": "2.7.12",
"camelVersion": "3.18.0",
"dependsOn": ["commons-jar"], // upstream repos
"downstreamDependents": ["proj-api"],
"nexusArtifact": "com.org:commons-jar",
"tektonPipeline": "commons-jar-pipeline",
"fixHistory": [
{ "date":"2026-05-01", "cves":["CVE-2024-1234"],
"change":"bumped jackson 2.15.2", "result":"success" }
]
}
```

First-run bootstrap: *if the file does not exist, the tool scans the pom.xml, detects modules, reads Spring Boot / Camel versions, and writes the file before calling the agent.*

4. Upgraded System Prompt (inject into fix_build_prompt)

Replace or extend your existing architect persona block with the following. Works with GPT-4o free, GPT-4o paid, Claude, or any instruction-following LLM.

```
You are a senior Java security architect specialising in Spring Boot and Apache Camel.
Your task is to fix ALL vulnerabilities listed below in a SINGLE consolidated pom.xml patch.

## ANALYSIS RULES — follow in this exact order
1. Read PROJECT MEMORY and understand project type (single-pom / multi-module).
2. Evaluate whether a Spring Boot version upgrade resolves the MAJORITY of CVEs.
If yes, include the upgrade as Step 1 of your plan before individual fixes.
3. For remaining CVEs after potential SB upgrade, determine for each:
a) Is it a direct dependency? -> bump version in dependencyManagement.
b) Is it a transitive dep? -> add on the pulling dep
AND add explicit safe version as direct dep.
4. Validate every proposed version exists in the internal Nexus before including it.
5. For multi-module projects: fix ONLY the root/parent pom unless a CVE is
module-specific and not inheritable from parent.
6. Never downgrade a dependency. Never change a non-vulnerable dependency.
7. Output a SINGLE unified diff / pom patch. No explanations outside XML comments.

## CONSTRAINTS
- Spring Boot BOM version governs managed deps — do not override managed versions
unless NexusIQ explicitly requires a higher version.
- Camel version must remain compatible with the Spring Boot version you choose.
Use the Camel Spring Boot compatibility matrix as your reference.
- ACS locked dependencies listed in LOCKS section must not be changed.

## OUTPUT FORMAT
Return only valid XML pom changes. Use inline XML comments .
If retrying after a build failure, address ONLY the error in the RETRY section.
```

---

5. Upgraded User Prompt (the message[] user content)

Inject the PROJECT MEMORY block immediately before the CVE list. This is what your fix_build_prompt assembles and writes to the JSON envelope.

```
## PROJECT MEMORY
Type : {type} // from project-registry.json
Spring Boot : {springBootVersion}
Camel : {camelVersion}
Depends on : {dependsOn} // upstream — already published to Nexus
Previous fixes : {fixHistory[-3:]}

## CVE LIST (from NexusIQ report)
{full_cve_list_extracted_from_pdf}
```

```
## ACS LOCKS (do not touch these)
{acs_locked_deps}

## CURRENT POM
{pom_content}

## RETRY SECTION (only present on retry attempt > 1)
Build failed with the following Maven errors:
{filtered_maven_error_lines}
Fix only the above errors. Do not re-apply already-correct changes.
```

6. Execution Flow with All Gaps Closed

Step	Action	Gap Closed
1	Bootstrap: scan repo, write project-registry.json if missing	G4, G5
2	Load registry + NexusIQ PDF into fix_build_prompt	G4
3	Inject System Prompt (Section 4) + User Prompt (Section 5)	G1, G2, G3, G5
4	Agent produces consolidated pom patch (single diff)	G1, G2, G3
5	validate-patch rejects non-pom / ACS-reverting changes	existing
6	mvn verify at repo root (multi-module: always root)	G5
7	On fail: inject Maven errors → retry (max 3)	existing
8	On pass: update fixHistory in project-registry.json	G4
9	Canary (1 repo) → human gate → batch (10) → human gate → all	G8
10	Poll Nexus API for upstream artifact before unblocking downstream repos	G6, G7

7. Nexus Availability Poll (for upstream deps)

After pushing commons-jar or any shared library, block downstream repos until Tekton publishes the artifact. Poll Nexus REST API — addresses G6, G7.

```
GET /service/rest/v1/search
?group=com.yourcompany
&name;=commons-jar
&version;={newVersion}

Poll every 30 seconds.
Timeout after 15 minutes → alert user, pause pipeline, wait for manual action.
Downstream repo status stays: WAITING_FOR_DEPENDENCY until artifact confirmed.
```

8. Repo Status State Machine

State	Meaning
PENDING	Queued, not started
IN_PROGRESS	Agent currently fixing
VERIFIED	mvn verify passed
PUSHED	Code pushed, Tekton triggered

WAITING_FOR_DEPENDENCY	Upstream artifact not yet in Nexus
DONE	All checks passed
FAILED	Build or verify failed — halts batch

9. Hard Rules — Never Violate

✗ Never push if mvn verify fails.

✗ Never touch production DB or block production queries.

✗ Never fix downstream repo before upstream artifact is confirmed in Nexus.

✗ Never push all repos simultaneously — always canary → batch → all with human gates.

✗ Never override ACS-locked dependencies.

✗ Never downgrade a dependency.

✗ Always run mvn verify from the root pom for multi-module projects.

✗ Always update project-registry.json after a successful fix.

✗ If any repo in a batch fails — stop the entire batch immediately.

10. Model Compatibility Note

Model	Works?	Note
GPT-4o (free)	**YES**	Use system+user role split already in copilot_agent.py
GPT-4o (paid)	**YES**	Same — higher rate limits, longer context
GPT-4.1 / o-series	**YES**	Better reasoning for constraint satisfaction
Claude Sonnet/Opus	**YES**	Drop-in via OpenAI-compatible endpoint or direct API
Gemini 2.5 Pro	**YES**	Via OpenAI-compatible layer or native API
Any local LLM	**MAYBE**	Needs strong instruction following; test on multi-module first

The system + user role split already implemented in copilot_agent.py means the prompt changes in Sections 4 and 5 work with any model that accepts OpenAI-style messages[] without any code change to copilot_agent.py.